1. Введение в Big Data

Цель лекции:
Сформировать системное понимание Big Data как **инженерной дисциплины**, где технологии служат инструментом для решения бизнес-проблем. Акцент сделан на:

- Глубоком анализе ограничений (5V),
- Связи архитектурных решений с метриками бизнеса,
- Реальных кейсах с техническими деталями (схемы, код, метрики),
- Ошибках, которые совершают начинающие разработчики.

---

1. Что такое Big Data? От абстракции к бизнес-ценности

Ключевой тезис:
Big Data — это не про «много данных», а про **преодоление ограничений**, которые делают традиционные системы (SQL-базы, Excel) непригодными для решения задач.

1.1. Пять «V» Big Data: инженерные последствия и реальные цифры

Каждый «V» — это **ограничение**, определяющее выбор технологий, процессов и метрик успеха.

Volume (Объем)

- **Техническое определение**:
  Данные, объем которых превышает **порог эффективности** для традиционных систем:
  - **1 ТБ** — предел для однопоточной обработки в разумные сроки (например, Pandas на сервере с 64 ГБ RAM),
  - **100 ТБ+** — требует распределенного хранения (HDFS, S3),
  - **1 ПБ+** — необходимость шардирования и оптимизации форматов.
- **Реальные примеры**:

Компания	Объем данных/день	Тип данных
CERN	1 ПБ	Данные с коллайдера
Крупный ритейлер	2.5 ПБ	Чеки, данные с IoT-датчиков

- **Инженерные последствия**:
  - **Форматы хранения**:
    - CSV: 1 ТБ данных → 10 ТБ операций ввода-вывода при чтении,
    - Parquet/ORC: колоночное хранение + сжатие → уменьшение объема на 75%, ускорение запросов в 10×.

```
# Пример: чтение 10 ГБ CSV vs. Parquet в PySpark
# CSV: 120 секунд, 15 ГБ RAM
```

```
df_csv = spark.read.csv("data.csv")
# Parquet: 12 секунд, 3 ГБ RAM
df_parquet = spark.read.parquet("data.parquet")
```

  - **Параллелизм**:
    - Разделение данных на партиции (например, по дате),
    - Настройка `spark.sql.shuffle.partitions` для избежания skew-проблем.
- **Ошибка новичка**:
  Попытка обработать 100 ГБ данных на локальной машине через Pandas → OOM (Out of Memory) и 8-часовой runtime вместо 5 минут в Spark.

Velocity (Скорость поступления)

- **Техническое определение**:
  Скорость генерации данных, требующая обработки с задержкой **ниже критического порога**:
  - **Батч-обработка (batch)**: задержка > 1 часа (например, ежедневная отчетность),
  - **Микробатч (micro-batch)**: задержка 1–60 сек (например, аналитика в реальном времени),
  - **Стриминг (streaming)**: задержка < 1 сек (например, fraud detection).
- **Реальные примеры**:

Сценарий	Скорость	Требуемая задержка
Финансовые транзакции	50 тыс./сек	< 100 мс
Данные с самолета	1 млн/сек	< 1 сек
Соцсети (лайки)	10 тыс./сек	< 5 сек

- **Инженерные последствия**:
  - **Буферизация**:
    - Kafka как «транспортная труба» с партициями и репликацией,
    - Настройка `retention.ms` для хранения данных в течение окна обработки.
  - **Потоковая обработка**:
    - Spark Streaming: обработка микробатчами (например, 30-секундные окна),
    - Flink: обработка событий по мере поступления (event-time processing).

```
# Пример: детекция аномалий в потоке транзакций через Flink
def detect_fraud(event):
    if event["amount"] > 1000 and event["velocity"] > 5:
        return True  # Подозрительная транзакция
    return False

stream.filter(detect_fraud).to("alerts_topic")
```

  - **Состояние (state)**:
    - Хранение контекста (например, скользящее среднее за 5 минут) в RocksDB или managed state.

- **Ошибка новичка**:
  Использование батч-обработки для fraud detection → мошенник успевает списать средства до обнаружения.

Variety (Разнообразие)

- **Техническое определение**:
  Наличие данных в **несовместимых форматах**, требующее гибких схем и преобразований:
  - **Структурированные**: SQL-таблицы, CSV,
  - **Полуструктурированные**: JSON, XML, Avro,
  - **Неструктурированные**: текст, изображения, видео, аудио.
- **Реальные примеры**:

Источник данных	Формат	Проблема преобразования
Логи веб-сервера	JSON	Вложенные поля (user.agent)
Отзывы клиентов	Текст	Сентимент-анализ
Скриншоты приложения	PNG	Обнаружение UI-элементов

- **Инженерные последствия**:
  - **Schema-on-read**:
    - Динамическая обработка JSON в Spark:
      ```
      df = spark.read.json("logs.json", multiLine=True)
      df.select("user_id", "event_type", "device.os")  # Доступ к вложенным полям
      ```
  - **Feature engineering для неструктурированных данных**:
    - Текст: TF-IDF, BERT-эмбеддинги,
    - Изображения: CNN (ResNet), объект-детекция (YOLO).
  - **Хранение**:
    - Data Lake (S3, ADLS) для «сырых» данных,
    - Data Warehouse (Snowflake, BigQuery) для очищенных таблиц.
- **Ошибка новичка**:
  Попытка загрузить все данные в SQL-базу → невозможность обработки вложенных JSON и рост стоимости.

Veracity (Достоверность)

- **Техническое определение**:
  Доля «грязных» данных, требующая **валидации и очистки**:
  - **Пропуски (missing values)**: >5% записей,
  - **Аномалии (outliers)**: значения за пределами физических ограничений (например, температура -50°C в тропиках),
  - **Смещение (bias)**: данные из ограниченного источника (только мобильные пользователи).
- **Реальные примеры**:

Сценарий	Проблема Veracity	Последствия
Датчики IoT	Сбойные показания	Ложные алерты о поломке
Онлайн-опросы	Спам-боты	Искажение аналитики
Геолокация	GPS-погрешность	Неверная маршрутизация

- **Инженерные последствия**:
  - **Data Validation**:
    - Great Expectations для проверки схемы и диапазонов:

```
expectation = {
    "expectation_type": "expect_column_values_to_be_between",
    "kwargs": {"column": "temperature", "min_value": -50, "max_value": 60}
}
```

  - **Outlier Detection**:
    - Статистические методы (Z-score, IQR),
    - ML-модели (Isolation Forest).
  - **Источники данных**:
    - Перекрестная проверка (например, сравнение данных с двух датчиков).
- **Ошибка новичка**:
  Обучение ML-модели на данных с 20% пропусков → точность 95% на трейне, 60% в production.

Value (Ценность)

- **Техническое определение**:
  Соотношение **затрат на обработку данных** к **бизнес-результату**:
  - **ROI =** (Прибыль от данных – Затраты на инфраструктуру) / Затраты на инфраструктуру,
  - **Порог эффективности**: ROI > 0.5 (50% прибыли на каждый вложенный доллар).
- **Реальные примеры**:

Проект	Затраты ($/год)	Прибыль ($/год)	ROI
Рекомендации стримингового сервиса	100 млн	1 млрд	9.0
Fraud detection (платёжный сервис)	50 млн	700 млн	13.0
Прогноз спроса (ритейл)	20 млн	100 млн	4.0

- **Инженерные последствия**:
  - **Фильтрация данных**:
    - Отказ от обработки данных с ROI < 0 (например, логи ошибок уровня DEBUG),
  - **Метрики успеха**:

    - Не accuracy модели, а влияние на бизнес (например, рост среднего чека на 5%).
  - **Cost Optimization**:
    - Spot-инстансы в облаке для батч-задач,
    - Tiered storage (горячие данные в S3, архивные в Glacier).
- **Ошибка новичка**:
  Создание сложной ML-модели для задачи, где простое правило «if-else» дает 90% эффективности.

> **Инсайт для инженеров**:
> Big Data — это **компромисс между 5V**. Например, для fraud detection критичны Velocity и Veracity, но Volume может быть умеренным. **Правило 80/20**: 80% усилий должно быть направлено на решение проблем, связанных с 2 критичными V для вашей задачи.

---

2. Типичные бизнес-кейсы: техническая реализация и метрики

Каждый кейс включает:

- **Бизнес-цель**,
- **Техническую архитектуру**,
- **Ключевые метрики**,
- **Ошибки при реализации**.

2.1. Рекомендательные системы

Бизнес-цель

Увеличить LTV (Lifetime Value) пользователя за счет персонализации.

Техническая архитектура

1. **Сбор данных**:
   - Клиентские SDK → Kafka (топик `user_events`),
   - Пример события:
     ```
     {
       "user_id": "u123",
       "item_id": "i456",
       "action": "click",
       "timestamp": "2023-10-05T14:30:00Z",
       "context": {"device": "mobile", "location": "NY"}
     }
     ```
2. **Feature Store**:
   - **Offline features**: история кликов за 30 дней (агрегация в Spark),
   - **Online features**: текущая сессия (хранилище в Redis),

- Инструменты: Feast, Tecton.

3. **Модели**:
   - **Collaborative filtering**: ALS в Spark MLlib для cold-start проблем,
   - **Deep learning**: Two-Tower модель (TensorFlow) для эмбеддингов пользователей и товаров.
4. **Деплой**:
   - **Offline**: ежедневное обновление рекомендаций в BigQuery,
   - **Online**: модель в TensorFlow Serving с latency < 50 мс.

Ключевые метрики

Метрика	Цель	Измерение
CTR (Click-Through Rate)	+15%	(Клики по рекомендациям / Показы)
Средний чек	+10%	До/после A/B-теста
Удержание (D7)	+5%	% пользователей, вернувшихся через 7 дней

Ошибки при реализации

- **Cold-start problem**:
  Нет рекомендаций для новых пользователей → решение: гибридная модель (популярные товары + контент-базированные рекомендации).
- **Data drift**:
  Сезонное изменение поведения (например, новогодние покупки) → решение: еженедельное ребалансирование модели.

2.2. Анализ логов и мониторинг систем

Бизнес-цель

Снизить MTTR (Mean Time To Repair) на 30% за счет автоматического обнаружения аномалий.

Техническая архитектура

1. **Сбор логов**:
   - Fluentd → Kafka (топик `raw_logs`),
   - Формат: JSON с полями `timestamp`, `service`, `log_level`, `message`.
2. **Обработка**:
   - Spark Streaming: агрегация ошибок за 5-минутные окна,
   - Flink: детекция аномалий через паттерны (например, резкий рост 500-х ошибок).
3. **Хранение**:
   - Elasticsearch для поиска по логам,

- Parquet в S3 для долгосрочного хранения.

4. **Визуализация**:
   - Grafana: дашборды с метриками (RPS, ошибка rate),
   - Алерты в Slack при превышении порога.

Ключевые метрики

Метрика	Цель	Измерение
MTTR	-30%	(Время от сбоя до исправления)
False Positive Rate	< 5%	(Ложные алерты / Общее число алертов)
Покрытие логов	100%	% сервисов с настроенным логгингом

Ошибки при реализации

- **Неправильные пороги алертов**:
  Слишком низкие пороги → шум в Slack,
  Решение: динамические пороги на основе исторических данных (например, 3σ от скользящего среднего).
- **Отсутствие контекста**:
  Алерт «Сервис упал» без информации о причине → решение: интеграция с distributed tracing (Jaeger).

---

3. Технологический стек: сравнение инструментов и выбор под задачу

3.1. Сравнение Hadoop, Spark и современных альтернатив

Критерий	Hadoop (MR + HDFS)	Spark	Dask	Ray
Тип обработки	Batch	Batch/Streaming/ML	Batch/ML	Distributed computing
Задержка	Высокая (минуты)	Низкая (мс)	Умеренная (секунды)	Очень низкая (мс)
Сложность настройки	Высокая	Средняя	Низкая	Средняя
ML-интеграция	Через Mahout (устаревший)	Встроенная (MLlib)	Scikit-learn	RLlib, Tune
Use case	Архивация данных	Универсальный выбор	Анализ на рабочей станции	Reinforcement Learning

Когда использовать Spark?

- **Да**, если:
  - Нужна обработка >1 ТБ данных,
  - Требуется интеграция ML (MLlib),

    - Задержка < 1 сек (Streaming).
- **Нет**, если:
    - Данные < 100 ГБ → Pandas/Dask,
    - Нужна обработка с задержкой < 10 мс → Flink.

Пример: миграция с Hadoop на Spark

Компания обрабатывала ETL на MapReduce за 2 часа. После перехода на Spark:

- Время обработки: 12 минут,
- Затраты на инфраструктуру: снизились на 40% (меньше узлов из-за in-memory обработки),
- Код упростился: 200 строк MapReduce → 50 строк PySpark.

3.2. Потоковая обработка: Kafka vs. Flink vs. Spark Streaming

Сравнение по ключевым параметрам

Параметр	Kafka Streams	Spark Streaming	Flink
Модель обработки	Micro-batch	Micro-batch	Event-time
Задержка	50–500 мс	500 мс – 2 сек	< 10 мс
State management	Базовое	Через checkpointing	Полное (managed state)
Легкость интеграции	Высокая	Средняя	Низкая
Use case	Простые трансформации	Аналитика в реальном времени	Fraud detection, IoT

Как выбрать?

- **Kafka Streams**:
    - Если обработка тесно связана с Kafka (например, enrichment сообщений),
    - Пример: добавление геолокации к транзакциям через MaxMind DB.
- **Spark Streaming**:
    - Если команда уже использует Spark для батч-обработки,
    - Пример: ежеминутная агрегация метрик для дашбордов.
- **Flink**:
    - Если критична задержка < 100 мс,
    - Пример: детекция мошенничества в банковских транзакциях.

Код: сравнение реализаций

```
# Spark Streaming: подсчет кликов за 1 минуту
stream = KafkaUtils.createDirectStream(ssc, ["clicks"], kafkaParams)
clicks = stream.map(lambda x: (x[1]["item_id"], 1))
clicks.reduceByKeyAndWindow(lambda a, b: a + b, 60, 60)

# Flink: детекция аномалий в реальном времени
stream.key_by("user_id").process(FraudDetector())
```

---

4. Разбор технологии: архитектура стриминговой платформы

4.1. Подробная схема архитектуры

```
[Клиентские приложения]
       ↓
[Kafka (ingestion)] → [Flink (real-time processing)] → [Cassandra (online features)]
       ↓
[Spark (batch processing)] → [S3 (Iceberg tables)] → [BigQuery (analytics)]
       ↓
[MLflow (model training)] → [Metaflow (deployment)] → [API Gateway]
```

Ключевые компоненты

1. **Ingestion Layer**:
   - **Kafka**: 10 тыс. партиций, репликация ×3,
   - **Schema Registry**: Avro для валидации событий.
2. **Real-time Processing**:
   - **Flink**: обработка 2 млн событий/сек,
   - **State**: RocksDB для хранения сессий пользователей.
3. **Storage**:
   - **Cassandra**: 500 узлов, latency < 10 мс для рекомендаций,
   - **S3 + Iceberg**: ACID-транзакции для исторических данных.
4. **ML Infrastructure**:
   - **Metaflow**: оркестрация пайплайнов (feature engineering → training → validation),
   - **Canary Deployment**: 1% трафика для тестирования новых моделей.

4.2. Как это решает бизнес-задачи

Снижение оттока

- **Проблема**: пользователи уходят после негативного опыта (буферизация, плохие рекомендации).

- **Решение**:
  - Real-time мониторинг буферизации через Flink,
  - Мгновенная замена контента при проблемах (например, переключение на локальный CDN).
- **Результат**: снижение оттока за счёт сокращения задержки.

Оптимизация трафика

- **Проблема**: значительная доля трафика уходит на повторную загрузку контента.
- **Решение**:
  - Прогнозирование популярного контента через AutoARIMA/AutoETS (StatsForecast),
  - Кэширование в локальных CDN (Akamai).
- **Результат**: сокращение затрат на трафик.

4.3. Уроки для разработчиков

1. **Data Contracts**:
   - Все события в Kafka проходят валидацию через Schema Registry → предотвращение поломок при изменении формата.
2. **Observability**:
   - Метрики для каждого компонента:
     - Kafka: lag, throughput,
     - Flink: checkpoint duration,
     - Cassandra: read latency.
3. **Cost Control**:
   - Spot-инстансы для батч-задач,
   - Автомасштабирование кластеров Spark на основе очереди задач.

---

5. Современные тренды и этические аспекты

5.1. Data Mesh: от централизованных команд к децентрализации

- **Суть**:
  Каждая бизнес-команда (например, «рекомендации», «платежи») отвечает за свои данные как за продукт.
- **Преимущества**:
  - Ускорение разработки (команда не ждет разрешения от централизованной Data Team),
  - Лучшее понимание данных (владелец знает бизнес-контекст).
- **Инструменты**:
  - **Domain Catalogs**: Amundsen, DataHub,
  - **Self-serve infrastructure**: Kubernetes-операторы для запуска Spark-джобов.

5.2. Data Lakehouse: объединение Lake и Warehouse

- **Проблема**:
  Data Lake (дешевое хранение) и Data Warehouse (быстрые запросы) требуют дублирования данных.
- **Решение**:
  - **Delta Lake**: ACID-транзакции поверх S3,
  - **Unity Catalog** (Databricks): централизованный контроль доступа.
- **Пример**:

```
-- Запрос к Delta Lake таблице с временными путешествиями
SELECT * FROM events VERSION AS OF 10 WHERE date = '2023-10-01'
```

5.3. Ethical Big Data: GDPR, приватность, предвзятость

- **Риски**:
  - **Приватность**: утечка данных → штрафы по GDPR (до 4% от выручки),
  - **Предвзятость**: ML-модели усиливают дискриминацию (например, кредитный скоринг).
- **Решения**:
  - **Anonymization**: замена PII (Personally Identifiable Information) на токены,
  - **Fairness Metrics**: проверка моделей на смещение (AIF360 библиотека).

5.4. Мультимодальность как расширение оси Variety

- **Суть**:
  В 2026 Variety — это не только текст и таблицы: изображения, аудио и видео обрабатывают вместе с табличными признаками, а не по отдельности.
- **Типовые модальности, форматы и обработка**:
  - Изображения (JPEG/PNG/WebP) — штатный декодер PyTorch TorchCodec 0.16 (torchvision-декодеры и nvJPEG/CUDA заменены; decord мёртв — последний коммит 07.2024),
  - Аудио (WAV/FLAC/MP3) — I/O через TorchCodec, признаки через librosa (torchaudio в фазе maintenance),
  - Видео (MP4/MKV) — PyAV; самый дорогой тип, в 10–100 раз дороже изображений.
- **Масштаб на уровне технологии**:
  - Модальность переводят в эмбеддинг замороженным энкодером (OpenCLIP 3.3.0 — дефолт открытых весов, SigLIP 2 `google/siglip2-*` в Transformers 5.17; аудио — CLAP/M2D-CLAP; видео — VideoMAE/VideoPrism/InternVideo2.5/V-JEPA 2),
  - Байты, эмбеддинги, индексы и версии хранят в одном формате (Lance; Blob v2: <16 КиБ inline, >2 МиБ отдельными файлами); поиск — по векторному индексу (LanceDB 0.38, Qdrant 1.19, pgvector 0.8.6, Milvus 3.0) с SQL-фильтрами,
  - Iceberg/Delta нативных блобов не имеют — используют модель «управляемых ссылок»; Parquet/Arrow остаётся табличным интермедиатом.
- **Ошибка новичка**: гонять декодирование медиа через Spark UDF; паттерн — декодировать/энкодить в Ray Data 2.58 или Daft 0.7.25, писать эмбеддинги и табличку в Lance/Parquet, джойнить в Spark 4.2/DuckDB.
- Подробно — **лекция 7**.

5.5. Синтетические данные как ответ на дефицит и приватность

- **Зачем синтетика**:
  Данных не хватает (редкие классы, чувствительные домены), а реальные ПДн нельзя публиковать. Синтетический датасет генерируют on-prem по схеме реального и выпускают наружу вместо самих данных.
- **Правовой контур (EDPB Guidelines 02/2026)**:
  - Приняты 08.07.2026, публичная консультация — до 30.10.2026,
  - Три теста, причём с точки зрения каждого, кто будет держать данные: **No Record Isolation** (singling-out) — нельзя выделить субъекта, **No Linkage** (linkability) — нельзя связать записи, **No Inference** (inference) — нельзя вывести атрибуты.
- **Отрезвляющий факт**:
  «Полностью анонимного» датасета не даёт ни один инструмент: в исследовании PEARL из 96 конфигураций DP-синтетики для образования прошли все проверки только 12. Типичные провалы — выпавшие группы (отчисленные) и разрушенный порядок учебных действий. Гейты приёмки должны быть задачно-зависимыми, а не только «по fidelity».
- **Пайплайн**:
  Удаление PII → псевдонимизация/обобщение → DP или синтез высокой точности → adversarial privacy audit → публикация только если гейты пройдены.
- Подробно — **лекция 8**.

5.6. Регуляторный контур 2025–26

- **152-ФЗ (ред. 26.07.2026)**:
  - ст. 6 ч. 1 п. 9 — обработка «в статистических или иных исследовательских целях» **при условии обязательного обезличивания**,
  - ст. 22 — уведомление РКН, ст. 18 ч. 5 — локализация,
  - ст. 13.1 (233-ФЗ, 2024) — режим «составов данных»: передача только государству, иностранным получателям запрещена; **для открытой межвузовской публикации не годится**.
- **GDPR** — базовое регулирование ЕС (штрафы до 4% от выручки); **AI Act** — регулирование ИИ по уровням риска.
- **Штрафы КоАП 13.11 (ред. 420-ФЗ, 2024)** за утечку персональных данных:

Масштаб утечки	Штраф
1–10 тыс. субъектов	3–5 млн ₽
10–100 тыс. субъектов	5–10 млн ₽
>100 тыс. субъектов	10–15 млн ₽
Повторно	1–3% выручки, мин. 20 млн ₽

- ⚠️ примечание 3 к ст. 13.11 адресует крупнейшие тиры операторам, не являющимся государственными/муниципальными органами и НКО, — вуз как НКО под них обычно не попадает, но уведомление, обезличивание и защита данных на нём остаются.

---

6. Заключение: Big Data как инженерная дисциплина

- **Главный принцип**:
  Технологии должны служить бизнесу, а не наоборот. Перед выбором инструмента ответьте:
  1. Какие 5V критичны для задачи?

2. Как измерить ROI этого проекта?
3. Какие метрики успеха примет бизнес?

- **Советы для начинающих**:
  - Начните с малого: прототипируйте на 10 ГБ данных перед переходом к распределенным системам,
  - Всегда измеряйте latency и cost на каждом этапе пайплайна,
  - Документируйте data contracts (форматы, SLA для задержек).
- **Следующая лекция**:
  Как выбрать между Pandas и Spark? Разберем кейсы, где однопоточные решения эффективнее распределенных.

Практическое задание

1. **Анализ 5V для кейса**:
   - Возьмите датасет NYC Taxi Trips,
   - Оцените его по 5V:
     - Volume: сколько займет 1 год данных в CSV/Parquet?
     - Velocity: сколько записей генерируется в час?
     - Variety: какие типы данных присутствуют (координаты, время)?
     - Veracity: найдите аномалии (поездки на 0 км, отрицательные суммы),
     - Value: как такси-компания может заработать на этих данных?
2. **Сравнение инструментов**:
   - Обработайте 10 ГБ данных через Pandas, Dask и PySpark,
   - Замерьте время выполнения, потребление RAM,
   - Составьте таблицу сравнения.

Вопросы для самопроверки

1. Почему Parquet ускоряет запросы к данным? Приведите пример с цифрами.
2. Как Flink обрабатывает late-arriving данные? Нарисуйте схему с окнами.
3. В чем разница между schema-on-read и schema-on-write? Как это влияет на ETL?
4. Как измерить ROI проекта по fraud detection? Напишите формулу.
5. Почему для online-хранилища фич выбирают Cassandra, а не DynamoDB? Какие метрики это объясняют?

Дополнительные материалы

- **Видео**:

    - Data Mesh: Decentralizing the Data Platform
- **Инструменты для практики**:
    - Databricks Community Edition (бесплатный Spark-кластер),
    - Confluent Cloud (Kafka в облаке).
- **Книги**:
    - *Streaming Systems* (Tyler Akidau) — глава 3: «Time and Windows»,
    - *Designing Data-Intensive Applications* (Martin Kleppmann) — глава 12: «Batch and Stream Processing».